

中国科学院大学

《计算机网络(研讨课)》实验报告

姓名 寇逸欣 学号 2023K8009922004 专业 计算机科学与技术
实验项目编号 05 实验名称 生成树机制实验

一、实验内容

本实验主要包括以下内容：

1. 基于已有代码，实现生成树运行机制，对于给定拓扑 (four_node_ring.py)，计算输出相应状态下的最小生成树拓扑
2. 自己构造一个不少于 7 个节点，冗余链路不少于 2 条的拓扑，节点和端口的命名规则可参考 four_node_ring.py，使用 stp 程序计算输出最小生成树拓扑

二、实验流程

本实验需要完成的是 stp.c 文件中的 stp_handle_config_packet 函数。这个函数的作用是根据接收到的 config 包，更新端口的状态，并且重新计算生成树的结构。本次实验是静态生成树，不用考虑拓扑变动下的生成树重构、如何与交换机转发学习共存以及如何快速构建生成树等问题。

首先，我们需要实现一个函数，用于比较两个 config 包的优先级，以确定哪个 config 包更优。比较的规则如下：

1. 如果两者认为的根节点 ID 不同，则根节点 ID 小的一方优先级高
2. 如果两者到根节点的开销不同，则开销小的一方优先级高
3. 如果两者到根节点的上一跳节点不同，则上一跳节点 ID 小的一方优先级高
4. 如果两者到根节点的上一跳端口不同，则上一跳端口 ID 小的一方优先级高

代码实现如下：

```
static int stp_tuple_cmp(u64 r1, u32 c1, u64 s1, u16 p1, u64 r2, u32 c2, u64 s2, u16 p2)
{
    if (r1 != r2) return (r1 < r2) ? -1 : 1;
    if (c1 != c2) return (c1 < c2) ? -1 : 1;
    if (s1 != s2) return (s1 < s2) ? -1 : 1;
    if (p1 != p2) return (p1 < p2) ? -1 : 1;
    return 0;
}
```

接下来，我们需要实现函数 stp_recompute 来重新计算生成树的结构。这个函数会遍历所有端口，找出每个节点到根节点的最优路径，并更新端口的状态。代码实现如下：

```
static void stp_recompute(stp_t *stp)
{
    // 1) 选根端口：只在“对端更优”的端口中选
    stp_port_t *best_port = NULL;
```

```

u64 best_root = 0;
u32 best_cost = 0;
u64 best_sw = 0;
u16 best_pid = 0;

for (int i = 0; i < stp->nports; i++) {
    stp_port_t *p = &stp->ports[i];
    if (p->designated_switch == stp->switch_id) continue;

    u64 r = p->designated_root;
    u32 c = p->designated_cost + p->path_cost;
    u64 s = p->designated_switch;
    u16 pid = p->designated_port;

    if (!best_port || stp_tuple_cmp(r, c, s, pid, best_root, best_cost, best_sw, best_pid) < 0) {
        best_port = p;
        best_root = r;
        best_cost = c;
        best_sw = s;
        best_pid = pid;
    }
}

// 2) 更新本桥的根信息
if (!best_port) {
    stp->designated_root = stp->switch_id;
    stp->root_path_cost = 0;
    stp->root_port = NULL;
} else {
    stp->designated_root = best_root;
    stp->root_path_cost = best_cost;
    stp->root_port = best_port;
}

// 3) 刷新各端口的 designated 信息
for (int i = 0; i < stp->nports; i++) {
    stp_port_t *q = &stp->ports[i];

    if (stp->root_port && q->port_id == stp->root_port->port_id)
        continue;

    u64 my_r = stp->designated_root;
    u32 my_c = stp->root_path_cost;
    u64 my_s = stp->switch_id;
    u16 my_p = q->port_id;

    if (stp_tuple_cmp(my_r, my_c, my_s, my_p,
        q->designated_root, q->designated_cost,
        q->designated_switch, q->designated_port) < 0) {
        q->designated_root = my_r;
    }
}

```

```

        q->designated_cost = my_c;
        q->designated_switch = my_s;
        q->designated_port = my_p;
    }
}
}

```

最后,我们利用上面实现的函数,在 `stp_handle_config_packet` 函数中处理接收到的 config 包。代码实现如下:

```

static void stp_handle_config_packet(stp_t *stp, stp_port_t *p, struct stp_config *config)
{
    u64 recv_root_id = ntohll(config->root_id);
    u32 recv_root_path_cost = ntohl(config->root_path_cost);
    u64 recv_switch_id = ntohll(config->switch_id);
    u16 recv_port_id = ntohs(config->port_id);

    bool superior = stp_tuple_cmp(
        recv_root_id, recv_root_path_cost, recv_switch_id, recv_port_id,
        p->designated_root, p->designated_cost, p->designated_switch, p->designated_port
    ) < 0;

    if (superior) {
        p->designated_root = recv_root_id;
        p->designated_cost = recv_root_path_cost;
        p->designated_switch = recv_switch_id;
        p->designated_port = recv_port_id;

        stp_recompute(stp);

        stp_send_config(stp);
    } else {
        if (stp_port_is_designated(p)) {
            stp_port_send_config(p);
        }
    }
}

```

三、实验结果及分析

3.1 实验结果

对于四节点环形拓扑 (`four_node_ring.py`),运行 `stp` 程序后,输出的最小生成树拓扑如下:

```

NODE b1 dumps:
INFO: this switch is root.
INFO: port id: 01, role: DESIGNATED.
INFO:  designated ->root: 0101, ->switch: 0101, ->port: 01, ->cost: 0.
INFO: port id: 02, role: DESIGNATED.

```

```

INFO:   designated ->root: 0101, ->switch: 0101, ->port: 02, ->cost: 0.

NODE b2 dumps:
INFO: non-root switch, designated root: 0101, root path cost: 1.
INFO: port id: 01, role: ROOT.
INFO:   designated ->root: 0101, ->switch: 0101, ->port: 01, ->cost: 0.
INFO: port id: 02, role: DESIGNATED.
INFO:   designated ->root: 0101, ->switch: 0201, ->port: 02, ->cost: 1.

NODE b3 dumps:
INFO: non-root switch, designated root: 0101, root path cost: 1.
INFO: port id: 01, role: ROOT.
INFO:   designated ->root: 0101, ->switch: 0101, ->port: 02, ->cost: 0.
INFO: port id: 02, role: DESIGNATED.
INFO:   designated ->root: 0101, ->switch: 0301, ->port: 02, ->cost: 1.

NODE b4 dumps:
INFO: non-root switch, designated root: 0101, root path cost: 2.
INFO: port id: 01, role: ROOT.
INFO:   designated ->root: 0101, ->switch: 0201, ->port: 02, ->cost: 1.
INFO: port id: 02, role: ALTERNATE.
INFO:   designated ->root: 0101, ->switch: 0301, ->port: 02, ->cost: 1.

```

示意图如下:

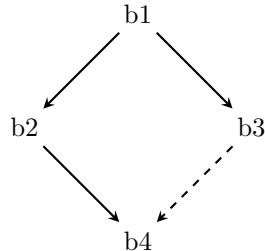


图 1: 四节点环形拓扑的最小生成树

八节点拓扑 (my_topology.py), 运行 stp 程序后, 输出的最小生成树拓扑如下:

```

NODE b1 dumps:
INFO: this switch is root.
INFO: port id: 01, role: DESIGNATED.
INFO:   designated ->root: 0101, ->switch: 0101, ->port: 01, ->cost: 0.
INFO: port id: 02, role: DESIGNATED.
INFO:   designated ->root: 0101, ->switch: 0101, ->port: 02, ->cost: 0.
INFO: port id: 03, role: DESIGNATED.
INFO:   designated ->root: 0101, ->switch: 0101, ->port: 03, ->cost: 0.

NODE b2 dumps:
INFO: non-root switch, designated root: 0101, root path cost: 1.
INFO: port id: 01, role: ROOT.
INFO:   designated ->root: 0101, ->switch: 0101, ->port: 01, ->cost: 0.

```

```

INFO: port id: 02, role: DESIGNATED.
INFO:  designated ->root: 0101, ->switch: 0201, ->port: 02, ->cost: 1.
INFO: port id: 03, role: DESIGNATED.
INFO:  designated ->root: 0101, ->switch: 0201, ->port: 03, ->cost: 1.

NODE b3 dumps:
INFO: non-root switch, designated root: 0101, root path cost: 1.
INFO: port id: 01, role: ROOT.
INFO:  designated ->root: 0101, ->switch: 0101, ->port: 02, ->cost: 0.
INFO: port id: 02, role: DESIGNATED.
INFO:  designated ->root: 0101, ->switch: 0301, ->port: 02, ->cost: 1.
INFO: port id: 03, role: DESIGNATED.
INFO:  designated ->root: 0101, ->switch: 0301, ->port: 03, ->cost: 1.

NODE b4 dumps:
INFO: non-root switch, designated root: 0101, root path cost: 2.
INFO: port id: 01, role: ROOT.
INFO:  designated ->root: 0101, ->switch: 0201, ->port: 02, ->cost: 1.
INFO: port id: 02, role: ALTERNATE.
INFO:  designated ->root: 0101, ->switch: 0301, ->port: 02, ->cost: 1.
INFO: port id: 03, role: DESIGNATED.
INFO:  designated ->root: 0101, ->switch: 0401, ->port: 03, ->cost: 2.

NODE b5 dumps:
INFO: non-root switch, designated root: 0101, root path cost: 1.
INFO: port id: 01, role: ROOT.
INFO:  designated ->root: 0101, ->switch: 0101, ->port: 03, ->cost: 0.
INFO: port id: 02, role: DESIGNATED.
INFO:  designated ->root: 0101, ->switch: 0501, ->port: 02, ->cost: 1.
INFO: port id: 03, role: DESIGNATED.
INFO:  designated ->root: 0101, ->switch: 0501, ->port: 03, ->cost: 1.

NODE b6 dumps:
INFO: non-root switch, designated root: 0101, root path cost: 2.
INFO: port id: 01, role: ROOT.
INFO:  designated ->root: 0101, ->switch: 0201, ->port: 03, ->cost: 1.
INFO: port id: 02, role: ALTERNATE.
INFO:  designated ->root: 0101, ->switch: 0501, ->port: 02, ->cost: 1.
INFO: port id: 03, role: DESIGNATED.
INFO:  designated ->root: 0101, ->switch: 0601, ->port: 03, ->cost: 2.

NODE b7 dumps:
INFO: non-root switch, designated root: 0101, root path cost: 2.
INFO: port id: 01, role: ROOT.
INFO:  designated ->root: 0101, ->switch: 0301, ->port: 03, ->cost: 1.
INFO: port id: 02, role: ALTERNATE.
INFO:  designated ->root: 0101, ->switch: 0501, ->port: 03, ->cost: 1.
INFO: port id: 03, role: DESIGNATED.
INFO:  designated ->root: 0101, ->switch: 0701, ->port: 03, ->cost: 2.

```

```

NODE b8 dumps:
INFO: non-root switch, designated root: 0101, root path cost: 3.
INFO: port id: 01, role: ROOT.
INFO:   designated ->root: 0101, ->switch: 0401, ->port: 03, ->cost: 2.
INFO: port id: 02, role: ALTERNATE.
INFO:   designated ->root: 0101, ->switch: 0601, ->port: 03, ->cost: 2.
INFO: port id: 03, role: ALTERNATE.
INFO:   designated ->root: 0101, ->switch: 0701, ->port: 03, ->cost: 2.

```

示意图如下：

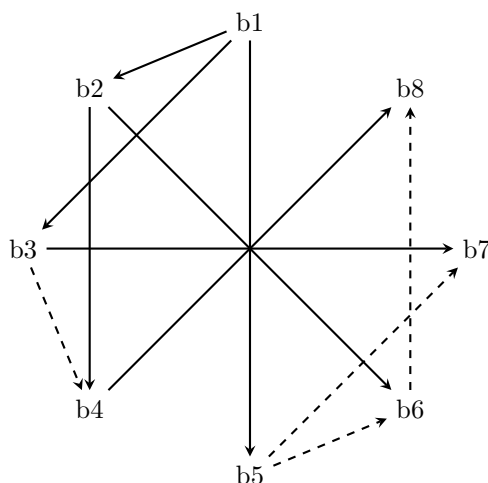


图 2: 八节点拓扑的最小生成树

3.2 结果分析

我们以四节点的环形拓扑为例进行分析。最开始，所有节点都认为自己是根节点，所有端口都被置为指定端口。当 b1 发送它的 config 包时，b2 和 b3 收到后发现 b1 的根节点 ID 更小，因此更新了自己的根节点信息，并将连接 b1 的端口设为根端口。随后，b2 和 b3 也发送了各自的 config 包，b4 收到后发现通过 b2 或 b3 到达 b1 的路径开销相同，但由于 b2 的交换机 ID 更小，因此选择通过 b2 作为根端口。最终，b4 通过比较收到的 config 包，确定连接 b3 的端口为其他端口，从而形成了最小生成树。

四、DEBUG 分析

在调试过程中，我遇到了一些问题，主要集中在以下几个方面：

1. 计算路径开销的时候，错误的多加了端口的路径开销，导致计算结果不正确。通过对比 reference 生成的正确结果，发现应该只加上收到的 config 包中的路径开销。
2. 在比较 config 包优先级时，忘记考虑上一跳端口的优先级，导致在某些情况下无法正确选择最优路径。通过补充比较逻辑，解决了这个问题。